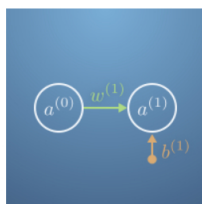


1. In this exercise we'll look in more detail about back-propagation, using the chain rule, in order to train our neural networks.

1 point

Let's look again at our two-node network.



Recall the activation equations are,

$$a^{(1)} = \sigma(z^{(1)})$$

$$z^{(1)} = w^{(1)}a^{(0)} + b^{(1)}.$$

Where we've introduced $z^{(1)}$ as the weighted sum of activation and bias.

We can formalise how good (or bad) our neural network is at getting the desired behaviour. For a particular input, x , and desired output y , we can define the *cost* of that specific *training example* as the square of the difference between the network's output and the desired output, that is,

$$C_k = (a^{(1)} - y)^2$$

Where k labels the training example and $a^{(1)}$ is assumed to be the activation of the output neuron when the input neuron $a^{(0)}$ is set to x

We'll go into detail about how to apply this to an entire set of training data later on. But for now, let's look at our toy example.

Recall our *NOT function* example from the previous quiz. For the input $x = 1$ we would like that the network outputs $y = 0$. For the starting weight and bias $w^{(1)} = 1.3$ and $b^{(1)} = -0.1$, the network actually outputs $a^{(1)} = 0.834$. If we work out the cost function for this example, we get

$$C_1 = (0.834 - 0)^2 = 0.696.$$

Do the same calculation for an input $x = 0$ and desired output $y = 1$. Use the code block to help you.

```
1 # First we set the state of the network
2 sigma = np.tanh
3 w1 = 1.3
4 b1 = -0.1
5
6 # Then we define the neuron activation.
7 def a1(a0) :
8     z = w1 * a0 + b1
9     return sigma(z)
10
11 # Experiment with different values of x below.
12 x = 1
13 a1(x)
14
```

Run

Reset

What is C_0 in this particular case? Give your result to 1 decimal place.

1.2

2. The cost function of a training set is the average of the individual cost functions of the data in the training set,

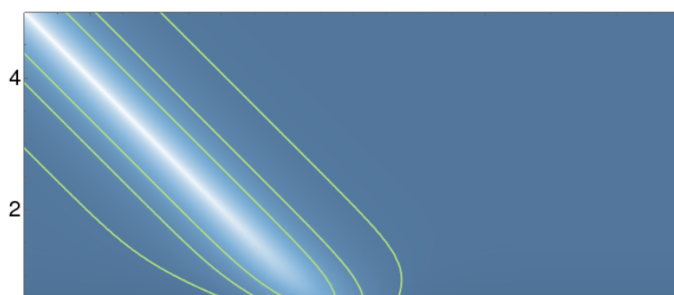
1 point

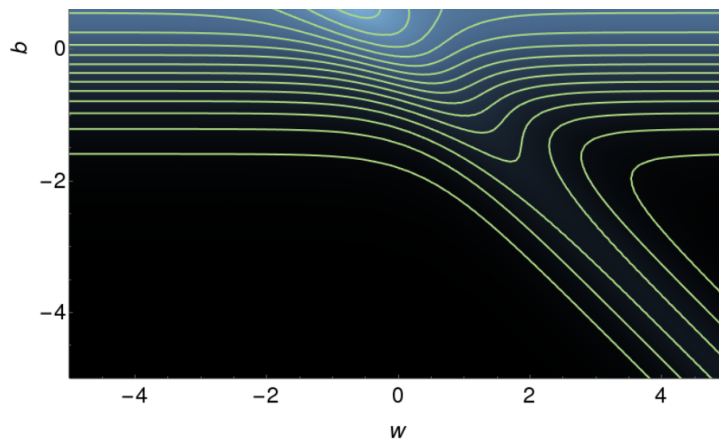
$$C = \frac{1}{N} \sum_k C_k,$$

where N is the number of examples in the training set.

For the NOT function we've been considering, where we have two examples in our training set, $(x = 0, y = 1)$ and $(x = 1, y = 0)$, the training set cost function is $C = \frac{1}{2}(C_0 + C_1)$.

Since our parameter space is 2D, $(w^{(1)}$ and $b^{(1)})$, we can draw the total cost function for this neural network as a contour map.





Here white represents low costs and black represents high costs.

Which of the following statements are true?

- ☒ The optimal configuration lies somewhere along the line $b = -w$.
- ☐ None of the other statements are true.
- ☐ There are many different local minima in this system.
- ☒ Descending perpendicular to the contours will improve the performance of the network.
- ☐ The optimal configuration lies along the line $b = 0$.

3. To improve the performance of the neural network on the training data, we can vary the weight and bias. We can calculate the derivative of the example cost with respect to these quantities using the chain rule.

1 point

$$\frac{\partial C_k}{\partial w^{(1)}} = \frac{\partial C_k}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial w^{(1)}}$$

$$\frac{\partial C_k}{\partial b^{(1)}} = \frac{\partial C_k}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial b^{(1)}}$$

Individually, these derivatives take fairly simple form. Go ahead and calculate them. We'll repeat the defining equations for convenience,

$$a^{(1)} = \sigma(z^{(1)})$$

$$z^{(1)} = w^{(1)}a^{(0)} + b^{(1)}$$

$$C_k = (a^{(1)} - y)^2$$

Select all true statements below.

- ☒ $\frac{\partial z^{(1)}}{\partial b^{(1)}} = 1$
- ☒ $\frac{\partial C_k}{\partial a^{(1)}} = 2(a^{(1)} - y)$
- ☐ $\frac{\partial z^{(1)}}{\partial b^{(1)}} = a^{(1)}$
- ☐ $\frac{\partial C_k}{\partial a^{(1)}} = (1 - y)^2$
- ☐ $\frac{\partial a^{(1)}}{\partial z^{(1)}} = \sigma$
- ☐ None of the other statements.
- ☐ $\frac{\partial z^{(1)}}{\partial w^{(1)}} = w^{(1)}$
- ☒ $\frac{\partial a^{(1)}}{\partial z^{(1)}} = \sigma'(z^{(1)})$
- ☒ $\frac{\partial z^{(1)}}{\partial w^{(1)}} = a^{(0)}$

$$\partial w^{(1)}$$

4. Using your answer to the previous question, let's see it implemented in code.

1 point

The following code block has an example implementation of $\frac{\partial C_k}{\partial w^{(1)}}$. It is up to you to implement $\frac{\partial C_k}{\partial b^{(1)}}$.

Don't worry if you don't know exactly how the code works. It's more important that you get a feel for what is going on.

We will introduce the following derivative in the code,

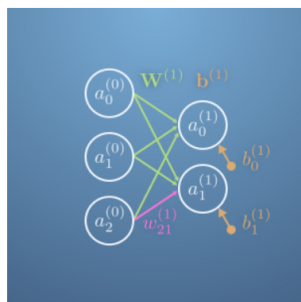
$$\frac{d}{dz} \tanh(z) = \frac{1}{\cosh^2 z}.$$

Complete the function 'dCdb' below. Replace the ??? towards the bottom, with the expression you calculated in the previous question.

```
1 # First define our sigma function.
2 sigma = np.tanh
3
4 # Next define the feed-forward equation.
5 def a1(w1, b1, a0):
6     z = w1 * a0 + b1
7     return sigma(z)
8
9 # The individual cost function is the square of the difference between
10 # the network output and the training data output.
11 def C(w1, b1, x, y):
12     return (a1(w1, b1, x) - y)**2
13
14 # This function returns the derivative of the cost function with
15 # respect to the weight.
16 def dCdw(w1, b1, x, y):
17     z = w1 * x + b1
18     dCda = 2 * (a1(w1, b1, x) - y) # Derivative of cost with activation
19     dadz = 1/np.cosh(z)**2 # derivative of activation with weighted sum z
20     dzdw = x # derivative of weighted sum z with weight
21     return dCda * dadz * dzdw # Return the chain rule product.
22
23 # This function returns the derivative of the cost function with
24 # respect to the bias.
25 # It is very similar to the previous function.
26 # You should complete this function.
27 def dCdb(w1, b1, x, y):
28     z = w1 * x + b1
29     dCda = 2 * (a1(w1, b1, x) - y)
30     dadz = 1/np.cosh(z)**2
31     """ Change the next line to give the derivative of
32     the weighted sum, z, with respect to the bias, b. """
33     dzdb = 1
34     return dCda * dadz * dzdb
35
36 """Test your code before submission:"""
37 # Let's start with an unfit weight and bias.
38 w1 = 2.3
39 b1 = -1.2
40 # We can test on a single data point pair of x and y.
```

5. Recall that when we add more neurons to the network, our quantities are upgraded to vectors or matrices.

1 point



$$\mathbf{a}^{(1)} = \sigma(\mathbf{z}^{(1)}),$$

$$\mathbf{z}^{(1)} = \mathbf{W}^{(1)}\mathbf{a}^{(0)} + \mathbf{b}^{(1)}$$

The individual cost functions remain scalars. Instead of becoming vectors, the components are summed over each output neuron.

$$C_k = \sum_i (a_i^{(1)} - y_i)^2$$

Note here that i labels the output neuron and is summed over, whereas k labels the training example.

The training data becomes a vector too,

$x \rightarrow \mathbf{x}$ and has the same number of elements as input neurons.

$y \rightarrow \mathbf{y}$ and has the same number of elements as output neurons.

This allows us to write the cost function in vector form using the modulus squared,

$$C_k = |\mathbf{a}^{(1)} - \mathbf{y}|^2.$$

Use the code block below to play with calculating the cost function for this network.

```

1 # Define the activation function.
2 sigma = np.tanh
3
4 # Let's use a random initial weight and bias.
5 W = np.array([[ -0.94529712, -0.2667356 , -0.91219181],
6               [ 2.05529992,  1.21797092,  0.22914497]])
7 b = np.array([ 0.61273249,  1.6422662 ])
8
9 # define our feed forward function
10 def a1 (a0) :
11     # Notice the next line is almost the same as previously,
12     # except we are using matrix multiplication rather than scalar multiplication
13     # hence the '@' operator, and not the '*' operator.
14     z = W @ a0 + b
15     # Everything else is the same though,
16     return sigma(z)
17
18 # Next, if a training example is,
19 x = np.array([0.1, 0.5, 0.6])
20 y = np.array([0.25, 0.75])
21
22 # Then the cost function is,
23 d = a1(x) - y # Vector difference between observed and expected activation
24 C = d @ d # Absolute value squared of the difference.
25

```

Run

Reset

For the initial weights and biases, what is the example cost function, C_k , when, $x = \begin{bmatrix} 0.7 \\ 0.6 \\ 0.2 \end{bmatrix}$ and

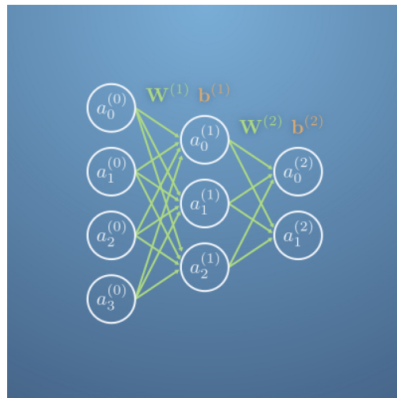
$$y = \begin{bmatrix} 0.9 \\ 0.6 \end{bmatrix}?$$

Give your answer to 1 decimal place.

1.8

6. Let's now consider a neural network with hidden layers.

1 point



Training this network is done by *back-propagation* because we start at the output layer and calculate derivatives backwards towards the input layer with the chain rule.

Let's see how this works.

If we wanted to calculate the derivative of the cost with respect to the weight or bias of the final layer, then this is the same as previously (but now in vector form):

$$\frac{\partial C_k}{\partial \mathbf{W}^{(2)}} = \frac{\partial C_k}{\partial \mathbf{a}^{(2)}} \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{W}^{(2)}}$$

With a similar term for the bias. If we want to calculate the derivative with respects to weights of the previous layer, we use the expression,

$$\frac{\partial C_k}{\partial \mathbf{W}^{(1)}} = \frac{\partial C_k}{\partial \mathbf{a}^{(2)}} \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{a}^{(1)}} \frac{\partial \mathbf{a}^{(1)}}{\partial \mathbf{z}^{(1)}} \frac{\partial \mathbf{z}^{(1)}}{\partial \mathbf{W}^{(1)}}$$

Where $\frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{a}^{(1)}}$ itself can be expanded to,

$$\frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{a}^{(1)}} = \frac{\partial \mathbf{a}^{(2)}}{\partial \mathbf{z}^{(2)}} \frac{\partial \mathbf{z}^{(2)}}{\partial \mathbf{a}^{(1)}}$$

This can be generalised to any layer,

$$\frac{\partial C_k}{\partial \mathbf{W}^{(i)}} = \frac{\partial C_k}{\partial \mathbf{a}^{(N)}} \underbrace{\frac{\partial \mathbf{a}^{(N)}}{\partial \mathbf{a}^{(N-1)}} \frac{\partial \mathbf{a}^{(N-1)}}{\partial \mathbf{a}^{(N-2)}} \cdots \frac{\partial \mathbf{a}^{(i+1)}}{\partial \mathbf{a}^{(i)}}}_{\text{from layer } N \text{ to layer } i} \frac{\partial \mathbf{a}^{(i)}}{\partial \mathbf{z}^{(i)}} \frac{\partial \mathbf{z}^{(i)}}{\partial \mathbf{W}^{(i)}}$$

By further application of the chain rule.

Choose the correct expression for the derivative,

$$\frac{\partial \mathbf{a}^{(j)}}{\partial \mathbf{a}^{(j-1)}}$$

Remembering the activation equations are,

$$a^{(n)} = \sigma(z^{(n)})$$

$$z^{(n)} = w^{(n)}a^{(n-1)} + b^{(n)}.$$

☐

$$\sigma'(\mathbf{z}^{(j)})\mathbf{W}^{(j-1)}$$

☐

$$\mathbf{W}^{(j)}\mathbf{a}^{(j)}$$

☒

$$\sigma'(\mathbf{z}^{(j)})\mathbf{W}^{(j)}$$

☐

$$\frac{\sigma'(\mathbf{z}^{(j)})}{\sigma'(\mathbf{z}^{(j-1)})}$$